



FORMATION INFORMATIQUE

Python

Les bases de Python, puis un cran plus loin : fichiers JSON, exceptions, classes.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Salut, c'est quoi Python ?](#)

- [A quoi ca sert ?](#)
- [Pourquoi commencer par Python ?](#)
- [Image mentale](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Mythe a casser](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

2. [Chapitre 2 - Installer et ecrire Python](#)

- [Ou ecrire le code ?](#)
- [Fichier '.py'](#)
- [Lancer](#)
- [La console interactive](#)
- [Virtualenv ? \(aperçu\)](#)
- [A toi](#)
- [Erreur classique](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

3. [Chapitre 3 - Ton premier programme](#)

- [print, c'est quoi ?](#)
- [f-strings \(pratique\)](#)
- [Commentaires](#)
- [input : demander quelque chose](#)
- [Nettoyer une reponse](#)
- [Erreur classique](#)
- [Exemple complet](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

4. [Chapitre 4 - Les variables](#)

- [Changer une valeur](#)
- [Noms clairs](#)
- [Plusieurs infos](#)
- [Constantes \(habitude\)](#)
- [Attention](#)
- [Exemple complet](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

5. Chapitre 5 - Les types (texte, nombre, vrai/faux)

- [Texte \(str\)](#)
- [Nombre entier \(int\)](#)
- [Nombre a virgule \(float\)](#)
- [Booleen \(bool\)](#)
- [Conversion](#)
- [type\(\)](#)
- [Pieges classiques](#)
- [None](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

6. Chapitre 6 - Les conditions (if)

- [elif](#)
- [Comparaisons](#)
- [and / or / not](#)
- [Conditions avec texte](#)
- [Operateur ternaire \(aperçu\)](#)
- [Exemple complet](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

7. Chapitre 7 - Les boucles

- [for + range](#)
- [while](#)
- [Boucler sur une liste](#)
- [break et continue](#)
- [while True \(motif utile\)](#)
- [Exemple : table de multiplication](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

8. Chapitre 8 - Les fonctions

- [Avec parametre](#)
- [return](#)
- [Valeur par default](#)
- [Plusieurs parametres](#)
- [Petite doc](#)
- [*args \(aperçu avance\)](#)
- [Pourquoi c'est bien](#)
- [Exemple complet](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

9. [Chapitre 9 - Les listes](#)

- [Modifier / longueur](#)
- [Ajouter / retirer](#)
- [Parcourir](#)
- [in](#)
- [Tranches \(slices\)](#)
- [Trier](#)
- [List comprehension \(avance soft\)](#)
- [Tuple \(cousin fige\)](#)
- [Ensemble set \(aperçu\)](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini défi](#)

10. [Chapitre 10 - Les dictionnaires](#)

- [Cle absente](#)
- [Parcourir](#)
- [Liste de dictionnaires](#)
- [Compter avec un dico](#)
- [JSON dans la tete](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini défi](#)

11. [Chapitre 11 - Lire et ecrire un fichier](#)

- [Ecrire](#)
- [Lire](#)
- [Lire ligne par ligne](#)
- [Modes utiles](#)
- [JSON \(plus avance, tres utile\)](#)
- [pathlib \(aperçu\)](#)
- [Erreurs classiques](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini défi](#)

12. [Chapitre 12 - Les modules \(import\)](#)

- [random](#)
- [datetime \(aperçu\)](#)
- [from ... import](#)
- [Creer ton module](#)
- [pip \(aperçu\)](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini défi](#)

13. [Chapitre 13 - Mini-projet : quiz terminal](#)

- [Version simple](#)
- [Version plus propre](#)
- [Ameliorations](#)
- [Version avec shuffle + JSON](#)
- [Criteres](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

14. [Chapitre 14 - Ce qu'il faut retenir \(bases\).](#)

- [En une minute](#)
- [Habitudes](#)
- [Erreurs classiques](#)
- [Carte mentale](#)
- [Suite immediate](#)
- [Mini check](#)

15. [Chapitre 15 - Les exceptions \(try / except\).](#)

- [Plusieurs erreurs](#)
- [else et finally](#)
- [raise \(lever une erreur\).](#)
- [Ne pas tout avaler](#)
- [Fichier manquant](#)
- [Exemple utile : saisie solide](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

16. [Chapitre 16 - Les classes \(objets\).](#)

- [Premiere classe](#)
- [Pourquoi c'est pratique](#)
- [Methode vs fonction](#)
- [__str__ \(affichage joli\).](#)
- [Heritage \(aperçu\).](#)
- [Dataclass \(tres moderne, aperçu\).](#)
- [Quand utiliser une classe ?](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

17. [Chapitre 17 - Atelier : lire les erreurs](#)

- [NameError](#)
- [TypeError](#)
- [ValueError](#)
- [IndexError / KeyError](#)
- [IndentationError / SyntaxError](#)
- [FileNotFoundError](#)
- [Methode](#)

- [Exercice 1](#)
- [Exercice 2](#)
- [Exercice 3](#)
- [Check final atelier](#)

18. [Chapitre 18 - Atelier : juste prix](#)

- [Version de base \(avec try\).](#)
- [Variantes](#)
- [Version objet \(bonus\).](#)
- [A toi](#)
- [Mini defi](#)

19. [Chapitre 19 - Atelier : organiser son code](#)

- [Version fonctions](#)
- [Pourquoi `if name == "main"` ?](#)
- [Version classe Question](#)
- [Decouper en fichiers \(idee\).](#)
- [Regle simple](#)
- [A toi](#)
- [Check](#)

20. [Quiz final - Teste-toi !](#)

- [Questions](#)
- [Reponses](#)
- [Score](#)

21. [Bravo.](#)

- [Ce que tu sais faire](#)
- [Mission finale](#)
- [La suite \(quand tu voudras\).](#)

Chapitre 1 - Salut, c'est quoi Python ?



Python sert à donner des ordres simples à l'ordinateur.

Python, c'est un langage pour donner des ordres à l'ordinateur.

Avec des phrases plus lisibles que beaucoup d'autres langages.

Tu écris une idée.

Python la traduit.

L'ordinateur exécute.

A quoi ça sert ?

Tu peux écrire de petits scripts utiles au quotidien, ou automatiser des trucs répétitifs qui te font perdre du temps. Ça t'apprend aussi la logique : conditions, boucles, et la façon de découper un problème. Plus tard, tu pourras aller vers la data, l'IA, les sites web ou les bots. Ici, on reste sur des bases solides. Puis on touche un peu à des notions plus avancées (erreurs, classes), sans devenir un manuel de 800 pages.

Pourquoi commencer par Python ?

Parce que c'est accueillant.

Moins de "bruit" autour.

Tu vois vite un résultat à l'écran.

Compare à d'autres langages : souvent plus de cérémonie avant le premier "Salut".

Avec Python, le premier programme tient en une ligne.

Image mentale

Tu es le chef.

Python est le messenger.

L'ordinateur exécute.

Si le message est flou, l'ordinateur se plaint (message d'erreur).

C'est normal. On apprend a lire ces messages.

Ce que tu vas savoir faire

Tu vas ecrire et lancer un programme. Tu manipuleras variables, types, conditions et boucles. Tu ecriras des fonctions avec parametres et `return`. Tu joueras avec listes, dictionnaires, et un peu de trucs plus fins. Tu liras et ecriras des fichiers (texte et JSON). Tu utiliseras des modules comme `random` ou `math`. Tu gereras des erreurs avec `try/except`, tu creeras une petite `class`, et tu feras un quiz et un jeu dans le terminal.

Comment lire ce livre

Lis un chapitre. Retape les exemples a la main. Change une valeur, puis relance. Fais le mini defi.

Si tu copies-colles sans reflechir, ca rentre moins.

Si tu tapes, ca rentre.

Mythe a casser

"Je dois tout comprendre avant d'ecrire."

Non.

Tu ecris. Ca casse. Tu corriges. Tu comprends mieux.

Allez. On installe, puis on code.

En vrai, sur le terrain

Dis a voix haute ce que tu veux que le programme fasse.

Ensuite seulement, ecris le code. Ca aide beaucoup.

Mini defi

Ecris sur papier 3 trucs que tu aimerais automatiser un jour
(ex : renommer des fichiers, calculer un score, faire un quiz).

On y reviendra.

Chapitre 2 - Installer et écrire Python

Avant de coder, il faut un Python qui marche.

Sur Windows, le plus simple :

1. Va sur le site officiel `python.org`
2. Télécharge la version récente
3. Installe (coche "Add Python to PATH" si tu vois l'option)
4. Ouvre un terminal et tape : `python --version`

Si tu vois un numéro de version, c'est bon.

Parfois c'est `py --version` qui marche mieux sur Windows.

Les deux existent. Garde celui qui répond.

Où écrire le code ?

Deux options débutant : **VS Code** (gratuit) avec l'extension Python, ou **IDLE** (fourni avec Python), plus basique mais ok. Plus tard tu peux aimer PyCharm ou autre. Pour ce livre, VS Code ou IDLE suffisent largement.

Fichier `.py`

Crée un dossier `mes-tests-python`.

Dedans, crée un fichier `salut.py`.

C'est ton programme.

Astuce : un dossier par projet.

Évite de tout mettre sur le Bureau en vrac.

Lancer

Dans le dossier du fichier :

```
python salut.py
```

Ou :

```
py salut.py
```

Si le terminal dit "introuvable", c'est souvent le mauvais dossier (tu n'es pas au bon endroit), ou le PATH pas coché à l'install.

La console interactive

```
python
```

Tu peux tester des lignes une par une.

Utile pour un calcul vite fait.

```
>>> 2 + 2
4
>>> exit()
```

`exit()` pour quitter.

Ou Ctrl+Z puis Entree sur Windows.

Virtualenv ? (aperçu)

Plus tard, les projets serieux utilisent un "environnement virtuel".

Idee : isoler les bibliotheques d'un projet.

```
python -m venv .venv
```

Puis on active, puis `pip install ...`.

Pour l'instant, tu n'en as pas besoin.

Mais le mot existe. Tu le reverras.

A toi

Verifie `python --version` (ou `py --version`). Cree le dossier et le fichier `salut.py`. On ecrira dedans au chapitre suivant.

Erreur classique

Tu lances `python salut.py` depuis le mauvais dossier.

Le terminal dit qu'il ne trouve pas le fichier.

Solution : `cd` vers le bon dossier, puis relance.

En vrai, sur le terrain

Ouvre le terminal. Tape la commande de version.

Note le resultat. Si ca marche, tu es pret.

Mini defi

Cree aussi `notes.md` dans ton dossier.

Ecris-y : version Python + outil choisi (VS Code / IDLE).

Chapitre 3 - Ton premier programme

Ouvre `salut.py` et écris :

```
print("Salut Python")
```

Lance :

```
python salut.py
```

Si tu vois `Salut Python`, bravo. C'est parti.

print, c'est quoi ?

`print(...)` affiche quelque chose à l'écran.

Le texte va entre guillemets.

```
print("Coucou")
print(2 + 2)
print("2 + 2 =", 2 + 2)
```

Tu peux afficher plusieurs choses séparées par des virgules.

Python ajoute un espace entre.

f-strings (pratique)

```
prenom = "Leo"
age = 12
print(f"Je m'appelle {prenom} et j'ai {age} ans")
```

Le `f` devant la chaîne laisse coller des variables avec `{...}`.

Plus lisible que coller avec `+` partout.

Commentaires

```
# Ceci est une note pour toi. Python l'ignore.
print("Je compte") # commentaire en fin de ligne
```

Écris des commentaires utiles, pas "ici j'affiche".

Plutôt : pourquoi tu fais un truc pas évident.

input : demander quelque chose

```
prenom = input("Ton prenom ? ")
print("Salut", prenom)
print(f"Enchante, {prenom} !")
```

`input` attend que tu ecrives puis Entree.

Attention : `input` renvoie toujours du **texte** (str).

Nettoyer une reponse

```
reponse = input("Ville ? ").strip()
```

`strip()` enleve les espaces avant/apres.

Utile. Les gens tapent souvent un espace sans le voir.

Erreur classique

Oublier les guillemets :

```
print(Salut) # erreur : Salut n'existe pas comme variable
```

Le message d'erreur pointe souvent la ligne. Lis-la.

Exemple complet

```
# mini presentation
prenom = input("Prenom ? ").strip()
hobby = input("Hobby ? ").strip()
print(f"Salut {prenom}. Cool pour {hobby}.")
print("Fin du programme.")
```

A toi

Demande le prenom et l'age (age en texte pour l'instant).

Affiche : `Bonjour, <prenom> ! Tu as <age> ans.`

Avec une f-string.

En vrai, sur le terrain

Retape l'exemple sans copier-coller.

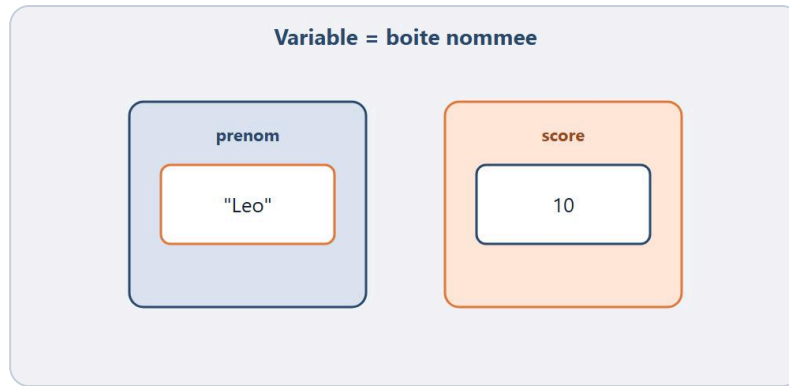
Change les phrases. Relance. Regarde.

Mini defi

Ajoute une 3e question (ville ou animal prefere).

Affiche une mini bio en 2 lignes.

Chapitre 4 - Les variables



Une variable, c'est une boîte avec un nom.

Une variable = une boîte avec un nom.

Tu ranges une valeur. Tu la reutilises plus tard.

```
age = 12
prenom = "Leo"
score = 0
```

Pas besoin de dire "c'est un nombre" ou "c'est du texte".

Python regarde la valeur et comprend.

Changer une valeur

```
score = 0
score = score + 1
score += 5 # raccourci : score = score + 5
print(score)
```

Le nom reste. Le contenu peut changer.

Noms clairs

Oui :

```
nombre_de_vies = 3
message_accueil = "Salut"
```

Non (ou alors on se perd) :

```
x1 = 3
a = "Salut"
```

En Python, on utilise souvent le **snake_case** : mots séparés par `_`.

C'est la convention. Suis-la. Ton futur toi te remerciera.

Plusieurs infos

```
a = 3
b = 5
total = a + b
print(total)
```

Tu peux aussi echanger :

```
x = 1
y = 2
x, y = y, x
print(x, y) # 2 1
```

Constantes (habitude)

Python n'a pas de vrai `const` comme certains langages.

Par convention, on écrit en MAJUSCULES ce qu'on ne veut pas changer :

```
MAX_ESSAIS = 5
```

Ce n'est pas bloqué techniquement. C'est un signal pour les humains.

Attention

```
# piège : écraser sans faire gaffe
score = 10
score = "dix" # autorise, mais souvent une mauvaise idée
```

Une variable peut changer de type. Ça ne veut pas dire que c'est clair.

Exemple complet

```
pseudo = "PixelFox"
niveau = 1
xp = 0

xp += 50
print(f"{pseudo} a {xp} xp")

if xp >= 50:
    niveau += 1
    xp -= 50
    print(f"Niveau up ! Niveau {niveau}")

print(f"Etat : niveau {niveau}, xp {xp}")
```

A toi

Crée une variable `prenom` et une variable `points` qui commence à 0. Ajoute 10 points, puis affiche le tout avec `print` ou une f-string.

En vrai, sur le terrain

Retape l'exemple joueur.

Change le pseudo et le seuil d'xp. Relance.

Mini defi

Invente un mini profil (jeu, sport, ecole).

3 variables minimum. Affiche une phrase complete.

Chapitre 5 - Les types (texte, nombre, vrai/faux)

Les types, c'est la nature de ce que tu ranges dans la boîte.

Texte (str)

```
phrase = "Bonjour"  
ville = 'Lyon' # simples ou doubles, au choix
```

Nombre entier (int)

```
vies = 3
```

Nombre à virgule (float)

```
prix = 19.99
```

Booleen (bool)

```
est_connecte = True  
a_fini = False
```

Attention : `True` / `False` avec majuscule.

Pas `true` comme en JS.

Conversion

`input` donne du texte. Pour calculer, convertis :

```
age_texte = input("Age ? ")  
age = int(age_texte)  
print("Dans 10 ans :", age + 10)
```

Versions courtes :

```
n = int(input("Nombre ? "))  
prix = float(input("Prix ? "))
```

Si la personne écrit "douze", `int(...)` plante.

Normal. Au chapitre exceptions, on apprendra à rattraper ça.

`type()`

```
print(type(3))      # int  
print(type("3"))    # str  
print(type(3.0))    # float  
print(type(True))   # bool
```


Utile pour comprendre ce que tu as vraiment.

Pieges classiques

```
print("3" + "1")    # "31" (colle du texte)
print(3 + 1)        # 4
# print("3" + 1)    # TypeError
```

Et :

```
print(bool(""))     # False (texte vide)
print(bool("0"))    # True (texte non vide)
print(bool(0))      # False
```

None

```
valeur = None # "pas de valeur" volontaire
```

Tu le croiseras avec les fonctions qui ne renvoient rien.

A toi

Demande deux nombres (via `input` + `int`).

Affiche leur somme, leur difference, leur produit.

En vrai, sur le terrain

Teste `type()` sur 4 valeurs differentes dans la console interactive.

Mini defi

Demande un prix en texte, convertis en `float`, ajoute 20% (TVA simple), affiche le TTC.

Chapitre 6 - Les conditions (if)

Les conditions, c'est "si ... alors ... sinon ...".

```
age = 15

if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

L'indentation (decalage) est obligatoire en Python.

En general : 4 espaces.

Sans ca, Python refuse. Ce n'est pas du decoratif.

elif

```
note = 14

if note >= 16:
    print("Excellent")
elif note >= 10:
    print("Valide")
else:
    print("On revise")
```

`elif` = "sinon si".

Tu peux en chainer plusieurs.

Comparaisons

`==` teste l'egalite (valeur). `!=` veut dire different. Tu as aussi `>`, `<`, `>=` et `<=`. Attention : `=` assigne. `==` compare.

and / or / not

```
ticket = True
vip = False

if ticket or vip:
    print("Entre")

if ticket and vip:
    print("Acces premium")

if not vip:
    print("Pas VIP")
```

Conditions avec texte

```
mdp = input("Mot de passe ? ").strip()
if mdp == "python123":
    print("OK")
else:
    print("Refuse")
```

Souvent on compare en minuscules :

```
if mdp.lower() == "python123":
    print("OK")
```

Operateur ternaire (aperçu)

```
age = 20
statut = "majeur" if age >= 18 else "mineur"
print(statut)
```

Pratique pour une petite decision. Pas obligatoire.

Exemple complet

```
score = int(input("Score ? "))
vies = int(input("Vies ? "))

if score >= 100 and vies > 0:
    print("Niveau suivant")
elif vies == 0:
    print("Game over")
else:
    print("Continue")
```

A toi

Demande un mot de passe.

Si c'est "python123", affiche "OK".

Sinon "Refuse".

Bonus : accepte aussi avec majuscules mixees via `.lower()`.

En vrai, sur le terrain

Change les seuils de l'exemple note. Relance avec 9, 10, 16.

Mini defi

Programme "meteo" : si temperature < 0 -> "froid", < 20 -> "doux", sinon "chaud".

Chapitre 7 - Les boucles

Une boucle = repeter sans tout retaper.

for + range

```
for i in range(1, 6):  
    print("Tour", i)
```

`range(1, 6)` = 1, 2, 3, 4, 5 (le 6 est exclu).

```
for i in range(5):      # 0..4  
    print(i)  
  
for i in range(0, 10, 2): # 0,2,4,6,8  
    print(i)
```

while

```
vies = 3  
while vies > 0:  
    print("Il reste", vies, "vies")  
    vies = vies - 1
```

Si tu oublies de diminuer `vies` : boucle infinie.

Ctrl+C pour arreter.

Boucler sur une liste

```
fruits = ["pomme", "banane", "kiwi"]  
for fruit in fruits:  
    print(fruit)
```

Avec index :

```
for i, fruit in enumerate(fruits, start=1):  
    print(i, fruit)
```

break et continue

```
for n in range(1, 10):  
    if n == 5:  
        break      # stop complet  
    if n % 2 == 0:  
        continue   # saute le reste du tour  
    print(n)
```

`break` = je sors.

`continue` = je passe au tour suivant.

while True (motif utile)

```
while True:
    cmd = input("Commande (q pour quitter) : ").strip().lower()
    if cmd == "q":
        break
    print("Tu as tape :", cmd)
```

Exemple : table de multiplication

```
n = 8
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

A toi

Affiche la table de 8 (de 8x1 a 8x10) avec `for` et `range`.

Puis une version `while`.

En vrai, sur le terrain

Fais une boucle qui compte de 10 a 1, puis affiche "Decollage".

Mini defi

Demande un mot de passe jusqu'a ce que ce soit le bon (max 3 essais).

Utilise `while` + `break`.

Chapitre 8 - Les fonctions

Une fonction = une recette réutilisable.

Tu la définis une fois. Tu l'appelles quand tu veux.

```
def dire_salut():  
    print("Salut !")  
  
dire_salut()  
dire_salut()
```

Avec paramètre

```
def dire_salut_a(prenom):  
    print(f"Salut {prenom}")  
  
dire_salut_a("Nora")  
dire_salut_a("Sam")
```

return

```
def double(n):  
    return n * 2  
  
resultat = double(4)  
print(resultat) # 8
```

Sans `return`, la fonction renvoie `None`.

`print` affiche. `return` renvoie une valeur au code appelant.

Ce n'est pas la même chose.

Valeur par défaut

```
def saluer(prenom="ami"):  
    print(f"Salut {prenom}")  
  
saluer()  
saluer("Lea")
```

Plusieurs paramètres

```
def moyenne(a, b):  
    return (a + b) / 2  
  
print(moyenne(12, 16))
```

Petite doc

Au-dessus du corps, tu peux laisser un commentaire clair :

```
def carre(n):  
    # Renvoie n au carre  
    return n * n
```

Plus tard tu verras aussi les "docstrings" (texte entre triples guillemets).

Meme idee : expliquer le role de la fonction.

***args (aperçu avancé)**

Parfois tu veux un nombre variable d'arguments :

```
def somme(*nombres):  
    total = 0  
    for n in nombres:  
        total += n  
    return total  
  
print(somme(1, 2, 3, 4)) # 10
```

Tu n'es pas obligé de l'utiliser tout de suite.

Mais tu le croiseras dans du vrai code.

Pourquoi c'est bien

Tu copies moins. Le code devient plus clair. C'est plus facile à corriger, parce que tu testes une brique à la fois.

Exemple complet

```
def note_sur_20(points, total):  
    if total == 0:  
        return 0  
    return round(points / total * 20, 1)  
  
print(note_sur_20(15, 20))  
print(note_sur_20(7, 10))
```

A toi

Ecris `moyenne(a, b)` qui renvoie la moyenne.

Puis `moyenne3(a, b, c)`.

Teste.

En vrai, sur le terrain

Ecris une fonction `est_pair(n)` qui renvoie True/False.

Teste avec 2, 3, 10.

Mini défi

Fonction `presenter(prenom, age)` qui retourne une phrase (pas print).

Tu printes le resultat a l'exterieur.

Chapitre 9 - Les listes



Une liste range plusieurs valeurs, une apres l'autre.

Une liste range plusieurs valeurs, une apres l'autre.

```
courses = ["pain", "lait", "oeufs"]
print(courses[0]) # pain
print(courses[-1]) # oeufs (dernier)
```

L'index commence a 0.

-1 = dernier element. Pratique.

Modifier / longueur

```
courses[1] = "lait d'avoine"
print(len(courses))
```

Ajouter / retirer

```
courses.append("beurre")
courses.insert(0, "eau")
dernier = courses.pop()
courses.remove("pain") # enleve la premiere occurrence
```

Parcourir

```
for item in courses:
    print("-", item)
```

in

```
if "pain" in courses:
    print("On a du pain")
```

Tranches (slices)

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4])    # 1,2,3
print(nums[:3])     # 0,1,2
print(nums[3:])     # 3,4,5
print(nums[::-1])   # inverse
```

Trier

```
notes = [12, 8, 15, 10]
print(sorted(notes))    # nouvelle liste
notes.sort()            # modifie sur place
notes.sort(reverse=True)
```

List comprehension (avance soft)

```
carres = [n * n for n in range(1, 6)]
print(carres) # [1, 4, 9, 16, 25]

pairs = [n for n in range(10) if n % 2 == 0]
```

C'est une boucle `for` en une ligne.

Lisible si c'est court. Sinon, garde une boucle classique.

Tuple (cousin fige)

```
point = (10, 20)
# point[0] = 11 # erreur : tuple non modifiable
```

Utile pour des paires fixes (coordonnees, resultat double...).

Ensemble set (aperçu)

```
tags = {"python", "debutant", "python"}
print(tags) # les doublons sautent
```

A toi

Liste de 4 jeux.

Ajoute-en un avec `append`.

Affiche tout avec une boucle.

Affiche aussi le dernier avec `[-1]`.

En vrai, sur le terrain

Fais une liste de notes. Calcule la moyenne avec une boucle.

Mini defi

A partir de `[1,2,3,4,5,6]` , cree la liste des pairs avec une comprehension.

Chapitre 10 - Les dictionnaires

Un dictionnaire = une fiche avec des clefs (pas seulement des numeros).

```
joueur = {
    "prenom": "Sam",
    "score": 120,
    "actif": True,
}

print(joueur["prenom"])
joueur["score"] = joueur["score"] + 10
joueur["niveau"] = 2 # nouvelle cle
```

Cle absente

```
print(joueur.get("niveau", 1))
```

Si "niveau" n'existe pas, tu obtiens 1.

Avec `joueur["niveau"]` direct : erreur si absent.

Parcourir

```
for cle, valeur in joueur.items():
    print(cle, "->", valeur)

print(joueur.keys())
print(joueur.values())
```

Liste de dictionnaires

```
equipe = [
    {"prenom": "Sam", "score": 120},
    {"prenom": "Lea", "score": 95},
]
print(equipe[0]["prenom"])

for j in equipe:
    print(f"{j['prenom']} : {j['score']}")
```

Compter avec un dico

```
votes = ["chat", "chien", "chat", "oiseau", "chat"]
compteur = {}
for mot in mots:
    compteur[mot] = compteur.get(mot, 0) + 1
print(compteur)
```

Pattern tres courant.

JSON dans la tete

Un dictionnaire Python ressemble beaucoup a du JSON.

Tu t'en serviras pour sauvegarder des configs, des scores, des API...

A toi

Cree un dico `livre` avec titre, pages, lu (True/False).

Affiche une phrase complete.

Ajoute une cle `auteur`.

En vrai, sur le terrain

Fais une petite "fiche eleve" (nom, moyenne, classe).

Modifie la moyenne. Reaffiche.

Mini defi

Liste de 3 produits `{nom, prix}`.

Affiche le total des prix avec une boucle.

Chapitre 11 - Lire et ecrire un fichier

Parfois tu veux garder des infos hors du programme.

Sinon tout disparaît à la fermeture.

Ecrire

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Salut\n")
    f.write("Deuxieme ligne\n")
```

`with` ferme le fichier proprement. Prends le réflexe.

Lire

```
with open("notes.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
    print(contenu)
```

Lire ligne par ligne

```
with open("notes.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(">", ligne.strip())
```

Modes utiles

Le mode `"w"` écrit (et efface). `"a"` ajoute à la fin. `"r"` lit seulement.

JSON (plus avancé, très utile)

```
import json

data = {"prenom": "Sam", "score": 120}

with open("joueur.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

with open("joueur.json", "r", encoding="utf-8") as f:
    charge = json.load(f)

print(charge["score"])
```

JSON = format texte standard pour échanger des données.

Parfait pour sauvegarder un score ou une config.

pathlib (aperçu)

```
from pathlib import Path

p = Path("notes.txt")
print(p.exists())
print(p.read_text(encoding="utf-8"))
```

Plus moderne que `open` seul pour certains cas.

Les deux cohabitent.

Erreurs classiques

Un mauvais dossier donne souvent "fichier introuvable". Oublier `encoding="utf-8"` foire les accents. Et le mode `"w"` écrase sans prévenir.

A toi

Ecris ton prénom dans `moi.txt`.

Relis le fichier et affiche-le.

Puis sauvegarde un petit dico dans `moi.json`.

En vrai, sur le terrain

Ajoute 3 lignes dans un journal `journal.txt` en mode `"a"`.

Relance 2 fois. Vérifie que ça s'allonge.

Mini défi

Programme qui demande un score, le sauve en JSON, le recharge, l'affiche.

Chapitre 12 - Les modules (import)

Un module = une boîte à outils déjà prête.

```
import math
print(math.sqrt(16))
print(math.pi)
```

random

```
import random

print(random.randint(1, 6)) # de 1 à 6 inclus
print(random.choice(["rouge", "vert", "bleu"]))
print(random.random())      # flottant entre 0 et 1
```

```
cartes = [1, 2, 3, 4, 5]
random.shuffle(cartes)
print(cartes)
```

datetime (aperçu)

```
from datetime import datetime
maintenant = datetime.now()
print(maintenant.strftime("%Y-%m-%d %H:%M"))
```

Utile pour dater un log ou un score.

from ... import

```
from math import pi, sqrt
print(pi, sqrt(9))
```

Créer ton module

Fichier `outils.py` :

```
def double(n):
    return n * 2
```

Autre fichier :

```
import outils
print(outils.double(5))
```

Pratique quand le projet grandit.

pip (aperçu)

Pour installer une bibliothèque tierce :

```
pip install requests
```

Puis :

```
# import requests # apres install
```

On ne l'utilise pas dans ce livre.

Mais tu dois connaître le réflexe : module standard d'abord, `pip` ensuite.

A toi

Simule un dé à 6 faces 5 fois avec une boucle.

Affiche aussi la somme des jets.

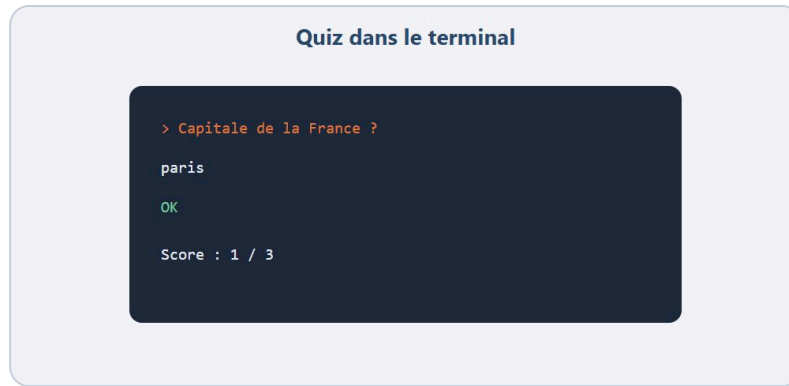
En vrai, sur le terrain

Mélange une liste de 5 prénoms avec `shuffle`, affiche le premier "tire au sort".

Mini défi

Petit "oracle" : tire une phrase au hasard dans une liste de 5 conseils.

Chapitre 13 - Mini-projet : quiz terminal



Mini projet : un petit quiz dans le terminal.

On assemble ce qu'on sait : variables, conditions, boucles, listes, dicos.

Version simple

```
score = 0

reponse = input("Capitale de la France ? ")
if reponse.strip().lower() == "paris":
    print("OK")
    score = score + 1
else:
    print("Non, c'etait Paris")

reponse = input("2 + 2 = ? ")
if reponse.strip() == "4":
    print("OK")
    score = score + 1
else:
    print("Non, 4")

reponse = input("Langage de ce livre ? ")
if reponse.strip().lower() == "python":
    print("OK")
    score = score + 1
else:
    print("Python !")

print("Score final :", score, "/ 3")
```

Version plus propre

```
questions = [
    {"q": "Capitale de la France ?", "a": "paris"},
    {"q": "2 + 2 = ?", "a": "4"},
    {"q": "Langage de ce livre ?", "a": "python"},
]

score = 0
for item in questions:
    rep = input(item["q"] + " ").strip().lower()
    if rep == item["a"]:
        print("OK")
        score += 1
    else:
        print("Non, reponse :", item["a"])

print(f"Score : {score} / {len(questions)}")
```

Améliorations

Tu peux mélanger l'ordre avec `random.shuffle`. Sauvegarde le meilleur score dans un fichier JSON. Ajoute 2 questions perso. Affiche un message selon le score (bien / moyen / a revoir).

Version avec shuffle + JSON

```
import json
import random
from pathlib import Path

questions = [
    {"q": "Capitale de la France ?", "a": "paris"},
    {"q": "2 + 2 = ?", "a": "4"},
    {"q": "Langage de ce livre ?", "a": "python"},
    {"q": "Couleur du ciel (souvent) ?", "a": "bleu"},
]

random.shuffle(questions)
score = 0
for item in questions:
    rep = input(item["q"] + " ").strip().lower()
    if rep == item["a"]:
        print("OK")
        score += 1
    else:
        print("Non :", item["a"])

print(f"Score : {score}/{len(questions)}")

fichier = Path("meilleur_score.json")
meilleur = 0
if fichier.exists():
    meilleur = json.loads(fichier.read_text(encoding="utf-8")).get("score", 0)

if score > meilleur:
    fichier.write_text(json.dumps({"score": score}, indent=2), encoding="utf-8")
    print("Nouveau record !")
else:
    print("Record actuel :", meilleur)
```

Criteres

Vise au moins 3 questions (idealement 5). Affiche le score a la fin. Compare de facon souple avec `strip` et `lower`.

En vrai, sur le terrain

Retape la version propre sans copier-coller.

Change les questions. Relance 2 fois.

Mini defi

Ajoute le melange + sauvegarde du record.

Chapitre 14 - Ce qu'il faut retenir (bases)

Tu as les bases. Ensuite : exceptions + classes, puis ateliers.

En une minute

Tu as vu `print`, `input` et les f-strings. Les variables se nomment souvent en `snake_case`. Les types courants sont `str`, `int`, `float`, `bool` et `None`. Tu sais brancher avec `if` / `elif` / `else`, et repeter avec `for` / `while` (plus `break` et `continue`). Les fonctions renvoient avec `return` (et tu as un aperçu de `*args`). Cote collections : listes (slices, comprehension), tuples, sets, et dictionnaires avec `.get`. Tu sais lire et écrire des fichiers texte et du JSON, et importer des modules (`math`, `random`, `json`, `datetime` ...).

Habitudes

Garde une indentation propre (4 espaces) et des noms clairs. Teste souvent. Lis les messages d'erreur sans paniquer. Prefere des petites fonctions. Pour les fichiers, prends le reflexe `with open(...)`.

Erreurs classiques

On oublie souvent `:` apres `if`, `for` ou `def`. Une mauvaise indentation fait tout planter. Comparer un nombre avec un texte (`"4"` vs `4`) donne des resultats bizarres. Un fichier "introuvable", c'est souvent le mauvais dossier. Et deux classiques : confondre `=` et `==`, ou confondre `print` et `return`.

Carte mentale

D'abord les donnees (variables, listes, dicos). Puis les decisions (`if`). Ensuite les repetitions (boucles). Puis l'organisation (fonctions, modules). Enfin la memoire (fichiers / JSON).

Suite immediate

Deux chapitres un cran au-dessus : gerer les plantages avec `try/except`, puis modeliser avec une `class`. Puis ateliers + quiz.

Mini check

Sans regarder : ecris de memoire un `if`, un `for`, une fonction avec `return`.

Si ca vient, tu es pret pour la suite.

Chapitre 15 - Les exceptions (try / except)



try : j'essaie. except : je rattrape.

Parfois le programme plante.

Pas grave. On peut rattraper.

Exemple classique : l'utilisateur tape une lettre au lieu d'un nombre.

```
texte = input("Nombre ? ")
try:
    n = int(texte)
    print("Tu as choisi", n)
except ValueError:
    print("Ce n'est pas un nombre.")
```

try = j'essaie.

except = si ca casse de cette facon, je fais autre chose.

Plusieurs erreurs

```
try:
    a = int(input("a ? "))
    b = int(input("b ? "))
    print(a / b)
except ValueError:
    print("Il faut des nombres.")
except ZeroDivisionError:
    print("Division par zero interdite.")
```

else et finally

```
try:
    n = int(input("Nombre ? "))
except ValueError:
    print("Nope")
else:
    print("OK, n =", n)  # seulement si pas d'erreur
finally:
    print("Toujours execute")
```

finally sert souvent a "nettoyer" (fermer un truc), meme en cas d'erreur.

raise (lever une erreur)

```
def age_valide(age):
    if age < 0:
        raise ValueError("Age negatif impossible")
    return age

print(age_valide(12))
# print(age_valide(-1)) # plante volontairement
```

Utile dans tes fonctions : signaler un probleme clair.

Ne pas tout avaler

Mauvais reflexe :

```
try:
    faire_un_truc()
except:
    pass # silence total = tu ne vois plus rien
```

Mieux : attraper l'erreur precise, ou au moins afficher un message.

Fichier manquant

```
from pathlib import Path

p = Path("secret.txt")
try:
    print(p.read_text(encoding="utf-8"))
except FileNotFoundError:
    print("Fichier introuvable. On continue.")
```

Exemple utile : saisie solide

```
def demander_int(message):
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Un entier, s'il te plait.")

age = demander_int("Age ? ")
print("Age =", age)
```

Tu peux reutiliser ca partout.

A toi

Reprends le juste prix (ou un `int(input(...))`).

Entoure la conversion avec `try/except` .

Si erreur : message + redemande.

En vrai, sur le terrain

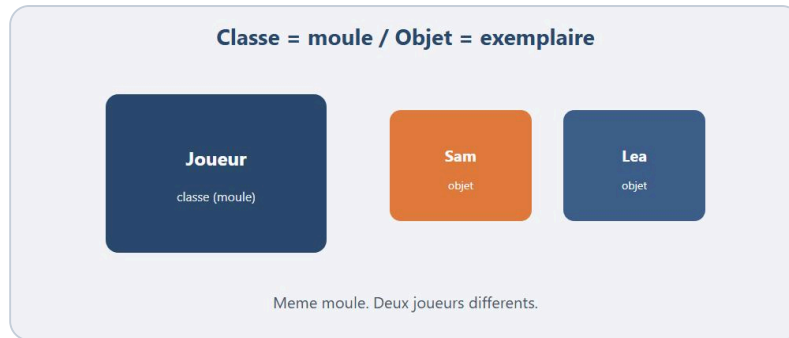
Fais planter volontairement `int("bonjour")` sans try.

Lis l'erreur. Puis protege.

Mini defi

Fonction `diviser(a, b)` qui renvoie le resultat ou `None` si division par zero, avec un message clair.

Chapitre 16 - Les classes (objets)



Une classe = un moule. Un objet = un exemplaire.

Jusqu'ici : fonctions + donnees separees.

Une classe, c'est coller les deux : "une fiche avec des actions".

Image mentale : un moule a cookies.

La classe = le moule.

Un objet = un cookie.

Premiere classe

```
class Joueur:
    def __init__(self, prenom, score=0):
        self.prenom = prenom
        self.score = score

    def gagner(self, points):
        self.score += points

    def presenter(self):
        return f"{self.prenom} : {self.score} pts"

j = Joueur("Sam")
j.gagner(10)
print(j.presenter())
```

`__init__` = constructeur. Appele a la creation.

`self` = "moi-meme", l'objet en cours.

Pourquoi c'est pratique

```
equipe = [Joueur("Sam"), Joueur("Lea", 50), Joueur("Noe")]
for j in equipe:
    j.gagner(5)
    print(j.presenter())
```

Chaque joueur a son propre score.

Pas besoin de 3 variables `score_sam`, `score_lea` ...

Methode vs fonction

Une fonction ressemble a `presenter(joueur)`. Une methode ressemble a `joueur.presenter()`. Meme idee, style different. Les methodes vivent avec les donnees.

`__str__` (affichage joli)

```
class Joueur:
    def __init__(self, prenom, score=0):
        self.prenom = prenom
        self.score = score

    def __str__(self):
        return f"{self.prenom} ({self.score})"

print(Joueur("Sam", 12))
```

Heritage (aperçu)

```
class Boss(Joueur):
    def gagner(self, points):
        # le boss gagne double
        self.score += points * 2
```

Une classe peut specialiser une autre.

Utile plus tard. Pas obligatoire maintenant.

Dataclass (tres moderne, aperçu)

```
from dataclasses import dataclass

@dataclass
class Produit:
    nom: str
    prix: float

p = Produit("Stylo", 1.5)
print(p.nom, p.prix)
```

Python ecrit une partie du boilerplate pour toi.

Tu le verras dans du code recent.

Quand utiliser une classe ?

Quand tu as plusieurs donnees liees et des actions dessus. Ou plusieurs "exemplaires" du meme genre (joueurs, produits, ennemis). Si c'est juste 2 fonctions et 1 variable : une fonction suffit. Pas besoin de classe partout.

A toi

Cree `Compte` avec `solde`.

Methodes `deposer(montant)` et `retirer(montant)`.

Refuse un retrait trop grand (message ou exception).

En vrai, sur le terrain

Cree 2 joueurs. Fais-les gagner des points. Affiche le plus fort.

Mini defi

Classe `Question` avec `texte` et `reponse` .

Methode `poser()` qui fait l'input et renvoie True/False.

Branche ca sur ton quiz.

Chapitre 17 - Atelier : lire les erreurs

Le terminal parle. Apprends à l'écouter.

Une erreur, c'est un GPS, pas une insulte.

NameError

Tu utilises un nom inconnu.

Souvent une faute de frappe : `scroe` au lieu de `score`.

TypeError

Tu mélanges mal les types.

Ex : `"3" + 1`

ValueError

Conversion impossible.

Ex : `int("bonjour")`

IndexError / KeyError

```
liste = [1, 2]
print(liste[5])          # IndexError
d = {"a": 1}
print(d["b"])            # KeyError
```

IndentationError / SyntaxError

Deux points `:` oubliés, ou décalage foireux.

Ou parenthèse non fermée.

FileNotFoundError

Mauvais chemin, mauvais dossier, fichier pas encore créé.

Méthode

Lis d'abord la **dernière** ligne de l'erreur (le type). Va au numéro de ligne. Corrige le plus petit truc, puis relance. Si besoin, entoure avec `try/except` - mais comprends d'abord.

Exercice 1

Casse volontairement ton quiz (enlève un `:`).

Lis l'erreur. Répare.

Exercice 2

Fais un `int(input(...))` et tape `abc`.

Puis protège avec `try/except` comme au chapitre 15.

Exercice 3

Ouvre un fichier qui n'existe pas.

Attrape `FileNotFoundError`. Affiche un message propre.

Check final atelier

Tu sais nommer 4 types d'erreurs courants, trouver la ligne fautive, et corriger sans tout reecrire.

Chapitre 18 - Atelier : juste prix

L'ordinateur choisit un nombre. Tu cherches.

Version de base (avec try)

```
import random

secret = random.randint(1, 20)
essais = 0
MAX = 6

while essais < MAX:
    essais += 1
    try:
        proposition = int(input(f"Essai {essais}/{MAX} - Nombre 1..20 ? "))
    except ValueError:
        print("Un nombre, merci.")
        continue

    if proposition < secret:
        print("Plus grand")
    elif proposition > secret:
        print("Plus petit")
    else:
        print(f"Bravo ! Trouve en {essais} essais")
        break
else:
    print(f"Perdu. C'etait {secret}")
```

Le `else` du `while` s'exécute si on ne `break` pas.

Pratique pour le message de défaite.

Variantes

Demande si on rejoue (o/n). Garde le meilleur score (moins d'essais) dans un JSON. Ajoute une difficulté : facile 1-10, normal 1-20, dur 1-50. En version objet, une classe `Partie` avec `secret`, `essais` et `jouer()`.

Version objet (bonus)

```
import random

class Partie:
    def __init__(self, maximum=20, max_essais=6):
        self.secret = random.randint(1, maximum)
        self.maximum = maximum
        self.max_essais = max_essais
        self.essais = 0

    def jouer(self):
        while self.essais < self.max_essais:
            self.essais += 1
            try:
                n = int(input(f"1..{self.maximum} ? "))
            except ValueError:
                print("Nombre invalide")
```

```
        continue
    if n == self.secret:
        print("Gagne")
        return True
    print("Plus grand" if n < self.secret else "Plus petit")
    print("Perdu :", self.secret)
    return False

if __name__ == "__main__":
    Partie().jouer()
```

A toi

Implemente au moins un max d'essais, un message de defaite, et une saisie protegee (`try/except`).

Mini defi

Ajoute le mode rejouer + sauvegarde du record.

Chapitre 19 - Atelier : organiser son code

Quand ca grandit, on coupe en fonctions (et parfois en classes).

Version fonctions

```
def poser_question(texte, bonne):
    rep = input(texte + " ").strip().lower()
    if rep == bonne:
        print("OK")
        return True
    print("Non, c'etait", bonne)
    return False

def main():
    score = 0
    if poser_question("Capitale de la France ?", "paris"):
        score += 1
    if poser_question("2 + 2 = ?", "4"):
        score += 1
    print("Score :", score)

if __name__ == "__main__":
    main()
```

Pourquoi `if __name__ == "__main__" ?`

Ca veut dire : "lance `main()` seulement si on execute ce fichier directement".

Utile quand tu importeras ce fichier ailleurs.

Version classe Question

```
class Question:
    def __init__(self, texte, reponse):
        self.texte = texte
        self.reponse = reponse

    def poser(self):
        rep = input(self.texte + " ").strip().lower()
        ok = rep == self.reponse
        print("OK" if ok else f"Non : {self.reponse}")
        return ok

def main():
    quiz = [
        Question("Capitale de la France ?", "paris"),
        Question("2 + 2 = ?", "4"),
        Question("Langage de ce livre ?", "python"),
    ]
    score = sum(1 for q in quiz if q.poser())
    print(f"Score : {score}/{len(quiz)}")
```



```
if __name__ == "__main__":  
    main()
```

Decouper en fichiers (idee)

Tu peux mettre les donnees ou la classe `Question` dans `questions.py`, la logique dans `jeu.py`, et le point d'entree dans `main.py`. Meme sur un petit projet, ca clarifie.

Regle simple

Une fonction / methode = une responsabilite.

`poser` pose.

`main` organise.

A toi

Reprends ton quiz. Decoupe avec au moins 2 fonctions. En bonus, passe en classe `Question`. En bonus 2, sauve le score en JSON.

Check

Si tu rouvres le fichier dans 2 semaines et tu comprends encore :
c'est bien organise.

Quiz final - Teste-toi !

Questions

Q1. `print("Salut")` sert a :

- A) Lire un fichier
- B) Afficher a l'ecran
- C) Installer Python

Q2. `input(...)` renvoie surtout :

- A) un int
- B) un str (texte)
- C) un bool

Q3. Pour convertir `"12"` en nombre :

- A) `str(12)`
- B) `int("12")`
- C) `print("12")`

Q4. Apres `if age >= 18:` on doit :

- A) Indenter le bloc
- B) Fermer avec `end`
- C) Mettre une virgule

Q5. `range(1, 4)` donne :

- A) 1, 2, 3, 4
- B) 1, 2, 3
- C) 0, 1, 2, 3

Q6. `return` dans une fonction :

- A) Affiche toujours
- B) Renvoie une valeur
- C) Efface la variable

Q7. Dans `["a", "b"]`, index de `"a"` :

- A) 1
- B) 0
- C) 2

Q8. Un dictionnaire associe :

- A) index numerique seulement
- B) cles -> valeurs
- C) uniquement des bool

Q9. `with open(...) as f:` c'est bien parce que :

- A) Ca colore le texte
- B) Ca gere l'ouverture/fermeture proprement
- C) Ca remplace `print`

Q10. `random.randint(1, 6)` :

- A) Tire un entier de 1 a 6
- B) Affiche toujours 6
- C) Cree un fichier

Q11. `True` et `False` s'ecrivent :

- A) true / false
- B) True / False
- C) TRUE / FALSE seulement

Q12. Indentation en Python :

- A) Optionnelle
- B) Importante (fait partie de la syntaxe)
- C) Interdite

Q13. `try/except` sert surtout a :

- A) Accelerer le code
- B) Rattraper une erreur sans tout planter
- C) Creer une variable

Q14. Dans une classe, `__init__` :

- A) Supprime l'objet
- B) Initialise l'objet a la creation
- C) Importe un module

Q15. `json.dump` sert a :

- A) Lire un JSON depuis un fichier
- B) Ecrire un objet Python en JSON dans un fichier
- C) Afficher du HTML

Q16. `liste[-1]` :

- A) Erreur toujours
- B) Dernier element
- C) Premier element

Q17. `score += 1` equivalent a :

- A) `score = score + 1`
- B) `score = 1`

- C) `print(score)`

Q18. `break` dans une boucle :

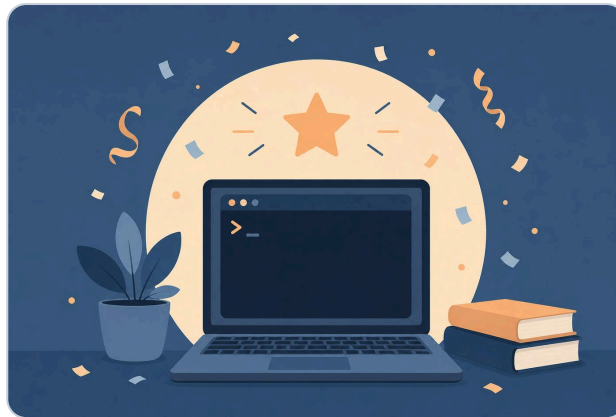
- A) Recommence le tour
- B) Sort de la boucle
- C) Quitte Python

Reponses

1 B, 2 B, 3 B, 4 A, 5 B, 6 B, 7 B, 8 B, 9 B, 10 A,
11 B, 12 B, 13 B, 14 B, 15 B, 16 B, 17 A, 18 B

Score

16-18 : nickel. 12-15 : bien, relis exceptions + classes + JSON. 8-11 : refais 5, 6, 8, 11, 15. Moins de 8 : reprends les chapitres 3 a 10 tranquillement.

Bravo.

Bravo. Tu as les bases de Python.

Tu as fini ce livre Python.

Pas "je connais tout".

Mais "je sais écrire un programme, le faire tourner, le corriger, et le structurer". C'est déjà énorme.

Ce que tu sais faire

Tu sais installer et lancer Python. Tu manipules variables, types, conditions et boucles. Tu écris des fonctions (paramètres, return, aperçu *args). Tu te débrouilles avec listes (slices, comprehensions) et dicos. Tu lis et écris des fichiers texte et du JSON. Tu utilises des modules (`random`, `math`, `json` ...). Tu gères les exceptions avec `try/except`. Tu as créé une première `class`. Et tu as fait un quiz et un juste prix.

Mission finale

Améliore ton juste prix en mode "produit mini". Limite à 6 essais avec un message de défaite. Protège la saisie. Propose de rejouer. Sauve le record en JSON. En bonus, une classe `Partie`.

La suite (quand tu voudras)

Tu pourras explorer `virtualenv` et `pip`, puis `Requests` pour parler à une API. Un peu de data (`pandas`) ou de web (`Flask/FastAPI`). Des tests automatiques avec `pytest`. Et lire du code des autres sur GitHub.

Mais là, respire.

Encore bravo.

Tu parles à l'ordinateur. Et maintenant, tu gères aussi quand il rale.

Tu as les briques. Maintenant, construis.